

human

Show me all the houses in Exeter that are within a 20 minute walk of a hospital. 20 mins means a buffer of 1600 around geometry of Hospital

host_agent

Calling tool send_message with args : {'target': 'planning_agent', 'task_description': 'Show me all the houses in Exeter that are within a 20 minute walk (buffer of 1600m) of a hospital. Steps needed to solve this geospatial query.'}

planning_agent

1. The general geographical area is Exeter, so there will be 1 entry for Exeter. 2. The entities to search for are houses in Exeter. 3. There is a condition to apply regarding proximity to hospitals, specifically within a 20-minute walk (buffer of 1600m) of a hospital. 4. We need to find all relevant houses that meet this condition.

output steps: ["Find Exeter 1 area", "Find houses in Exeter as many search results", "Find hospitals in Exeter as many search results", "Apply condition for houses within 1600m of hospitals"]

host_agent

Calling tool send_message with args : {'target': 'named_area', 'task_description': 'Find the area polygon for Exeter.'}

named_area_agent

Calling tool call_os_ngd with args : {'filters': None, 'bbox': None, 'polygon_or_point': True, 'street_address': None, 'filename': 'exeter_area_polygon'} Tool call_os_ngd returned result : ['Search results have been found. \n Artifact generated is : exeter_area_polygon,\n Description is : A geopandas dataframe containing named area data with no filters applied for the query (so remember no search is performed here so spurious entities present) Find the area polygon for Exeter.,\n count of records fetched : 98.\n ',] Calling tool send_message with args : {'target': 'data_analysis_agent', 'task_description': "Filter the artifact 'exeter_area_polygon' to find the specific polygon for Exeter. Save the filtered results as 'filtered_exeter_area_polygon'."}

data_analysis_agent

Calling tool generate_metadata_for_artifacts with args : {'artifact_names': ['exeter_area_polygon']} Tool Code Executor

```
def filter_exeter_area_polygon(data: list):
    gdf = data[0]
    # Filter for polygons specifically representing Exeter
    # Use name1_text for 'Exeter', and also check for relevant descriptiongroup/description
    filtered = gdf[
        (
            (gdf['name1_text'].str.lower() == 'exeter') |
            (gdf['name1_text'].str.lower().str.contains('exeter'))
        ) & (
            (gdf['descriptiongroup'].isin(['Settlement', 'Part Of Settlement'])) |
            (gdf['description'].isin(['City', 'Town', 'Suburban Area']))
        )
    ]
    summary = f"Filtered to {len(filtered)} polygon(s) for Exeter."
    artifact_name = 'filtered_exeter_area_polygon'
    artifact_description = 'GeoDataFrame containing only the polygon(s) representing Exeter, filtered by name and settlement/city/town/suburban area type.'
    return [summary, artifact_name, artifact_description, filtered]
```

Tool Code Executor

```
def filter_exeter_area_polygon(data: list):
    gdf = data[0]
    # Filter for polygons specifically representing Exeter
    # Use name1_text for 'Exeter', and also check for relevant descriptiongroup/description
    filtered = gdf[
        (
            (gdf['name1_text'].str.lower() == 'exeter') |
            (gdf['name1_text'].str.lower().str.contains('exeter'))
        ) & (
            (gdf['descriptiongroup'].isin(['Settlement', 'Part Of Settlement'])) |
            (gdf['description'].isin(['City', 'Town', 'Suburban Area']))
        )
    ]
    summary = f"Filtered to {len(filtered)} polygon(s) for Exeter."
    artifact_name = 'filtered_exeter_area_polygon'
    artifact_description = 'GeoDataFrame containing only the polygon(s) representing Exeter, filtered by name and settlement/city/town/suburban area type.'
    return [summary, artifact_name, artifact_description, filtered]
```

Output: ['Filtered to 1 polygon(s) for Exeter.',]

named_area_agent

The specific polygon for Exeter has been filtered and saved as 'filtered_exeter_area_polygon'. This GeoDataFrame contains only the polygon(s) representing Exeter, filtered by name and settlement/city/town/suburban area type.

host_agent

Calling tool send_message with args : {'target': 'buildings', 'task_description': "Find all houses in the area defined by 'filtered_exeter_area_polygon'. Filter for residential houses only."}

buildings_agent

Calling tool call_os_ngd with args : {'filters': None, 'bbox': 'filtered_exeter_area_polygon', 'polygon_or_point': True, 'street_address': None, 'filename': 'all_buildings_in_filtered_exeter_area.geojson'} Tool call_os_ngd returned result : ['Multiple search results have been found in multiple datasets. A summary of each is provided.\n Artifacts generated are : ['buildings_all_buildings_in_filtered_exeter_area.geojson\', \'buildingline_all_buildings_in_filtered_exeter_area.geojson\', \'buildingpart_all_buildings_in_filtered_exeter_area.geojson\'],\n Descriptions are : ["A geopandas dataframe containing building data with filters and bbox applied for the query Find all houses in the area defined by \'filtered_exeter_area_polygon\'. Filter for residential houses only. using filters None.", "A geopandas dataframe containing building line data with filters and bbox applied for the query Find all houses in the area defined by \'filtered_exeter_area_polygon\'. Filter for residential houses only. using filters None.", "A geopandas dataframe containing building part data with filters and bbox applied for the query Find all houses in the area defined by \'filtered_exeter_area_polygon\'. Filter for residential houses only. using filters None."],\n counts of records fetched are : [69169, 69342, 85480].\n ', [, ,]]

Calling tool send_message with args : {'target': 'data_analysis_agent', 'task_description': "From the artifact 'buildings_all_buildings_in_filtered_exeter_area.geojson', filter and return only those buildings that are houses. A house is defined as a building where buildinguse_addresscount_residential > 0 and buildinguse_addresscount_total = 1. Save the filtered results as 'residential_houses_in_filtered_exeter_area.geojson'."}

data_analysis_agent

Calling tool generate_metadata_for_artifacts with args : {'artifact_names': ['buildings_all_buildings_in_filtered_exeter_area.geojson']} Tool Code Executor

```
def filter_residential_houses(data: list):
    gdf = data[0]
    filtered = gdf[(gdf['buildinguse_addresscount_residential'] > 0) & (gdf['buildinguse_addresscount_total'] == 1)]
    filtered = filtered.copy()
    # Save to geojson
    filtered.to_file('residential_houses_in_filtered_exeter_area.geojson', driver='GeoJSON')
    summary = f"Filtered {len(filtered)} residential houses from {len(gdf)} total buildings."
    artifact_name = 'residential_houses_in_filtered_exeter_area.geojson'
    artifact_description = 'GeoDataFrame of buildings in the filtered Exeter area that are classified as houses (residential, single address).'
```

```
return [summary, artifact_name, artifact_description, filtered]
```

Output: ['Filtered 38848 residential houses from 69169 total buildings.',]

Filtered 38,848 residential houses from 69,169 total buildings in the filtered Exeter area. The resulting artifact contains only those buildings classified as houses, defined as having a residential address count greater than 0 and a total address count of 1. The filtered data is saved as 'residential_houses_in_filtered_exeter_area.geojson'.

buildings_agent

- 38,848 residential houses have been identified in the area defined by 'filtered_exeter_area_polygon'.
- These houses are filtered from the total buildings using the definition: `buildinguse_addresscount_residential > 0` and `buildinguse_addresscount_total = 1` (as per the standard definition of a house).
- The filtered results are saved in the artifact: `residential_houses_in_filtered_exeter_area.geojson`.

Artifact: - `residential_houses_in_filtered_exeter_area.geojson`: GeoDataFrame of buildings in the filtered Exeter area that are classified as houses (residential, single address).

host_agent

Calling tool `send_message` with args : `{'target': 'land_use_features', 'task_description': "Find all hospitals in the area defined by 'filtered_exeter_area_polygon'. Filter for hospitals only."}`

land_use_features_agent

Calling tool `call_os_ngd` with args : `{'filters': ['Hospital'], 'bbox': 'filtered_exeter_area_polygon', 'polygon_or_point': True, 'street_address': False, 'filename': 'hospitals_in_exeter'}` Tool `call_os_ngd` returned result : `['Multiple search results have been found in multiple datasets. A summary of each is provided.\n Artifacts generated are : ['landuse_hospitals_in_exeter'],\n Descriptions are : ["A geopandas dataframe containing land use data with filters and bbox applied for the query Find all hospitals in the area defined by 'filtered_exeter_area_polygon'. Filter for hospitals only. using filters ['Hospital']. Further Name filtering is available for this"],\n counts of records fetched are : [9].\n ', []]` I found 9 records of hospitals in the area defined by 'filtered_exeter_area_polygon'. The data is saved in the artifact named 'landuse_hospitals_in_exeter'.

host_agent

Calling tool send_message with args : {'target': 'plotting_agent', 'task_description': "Plot all houses from 'residential_houses_in_filtered_exeter_area.geojson' that are within a 1600m buffer of any hospital in 'landuse_hospitals_in_exeter'. Show the Exeter area boundary from 'filtered_exeter_area_polygon' for context."}

plotting_agent

Tool Code Executor

```
def plot_houses_within_1600m_of_hospitals(data: list):
    import geopandas as gpd
    import folium
    from shapely.geometry import Polygon
    from shapely.ops import unary_union
    import os

    # Unpack data
    houses = data[0]
    hospitals = data[1]
    exeter_area = data[2]

    # Ensure all CRS are the same and in EPSG:4326 for folium
    target_crs = 'EPSG:4326'
    if houses.crs != target_crs:
        houses = houses.to_crs(target_crs)
    if hospitals.crs != target_crs:
        hospitals = hospitals.to_crs(target_crs)
    if exeter_area.crs != target_crs:
        exeter_area = exeter_area.to_crs(target_crs)

    # Create a 1600m buffer around all hospital polygons (buffer in meters, so use a projected CRS first)
    projected_crs = 'EPSG:27700' # British National Grid for accurate meter buffering
    hospitals_proj = hospitals.to_crs(projected_crs)
    hospitals_buffer = hospitals_proj.buffer(1600)
    hospitals_buffer = gpd.GeoSeries(unary_union(hospitals_buffer), crs=projected_crs)
    hospitals_buffer = hospitals_buffer.to_crs(target_crs)

    # Filter houses that intersect with the buffer
    houses_in_buffer = houses[houses.intersects(hospitals_buffer.iloc[0])]

    # Prepare folium map centered on Exeter
    exeter_centroid = exeter_area.geometry.iloc[0].centroid
    m = folium.Map(location=[exeter_centroid.y, exeter_centroid.x], zoom_start=12, tiles='cartodbpositron')
```

```

# Plot Exeter area boundary
folium.GeoJson(
    exeter_area.geometry.iloc[0],
    name='Exeter Area',
    style_function=lambda x: {'color': 'black', 'weight': 2, 'fillOpacity': 0.05}
).add_to(m)

# Plot hospital buffer
folium.GeoJson(
    hospitals_buffer.iloc[0],
    name='1600m Hospital Buffer',
    style_function=lambda x: {'color': 'red', 'weight': 2, 'fillOpacity': 0.1}
).add_to(m)

# Plot hospitals
for _, row in hospitals.iterrows():
    folium.GeoJson(
        row.geometry,
        name=row.get('name1_text', 'Hospital'),
        style_function=lambda x: {'color': 'blue', 'weight': 2, 'fillOpacity': 0.2}
    ).add_to(m)

# Plot houses within buffer
folium.GeoJson(
    houses_in_buffer.geometry,
    name='Houses within 1600m of Hospital',
    style_function=lambda x: {'color': 'green', 'weight': 1, 'fillOpacity': 0.3}
).add_to(m)

folium.LayerControl().add_to(m)

# Save map
map_filename = 'houses_within_1600m_of_hospitals.html'
m.save(map_filename)

# Prepare summary
summary = f"There are {len(houses_in_buffer)} houses within 1600m of any hospital in Exeter. First 5 osid: {houses_in_buffer['osid'].head(5).tolist()}"
artifact_name = 'houses_within_1600m_of_hospitals_map'
artifact_description = 'A folium map showing all residential houses within 1600m of any hospital in Exeter, with the Exeter area boundary and hospital buff
return [summary, artifact_name, artifact_description, map_filename]

```

Output: ["There are 31188 houses within 1600m of any hospital in Exeter. First 5 osid: ['ed594210-3aa5-4f04-a1c2-479c54fcd967', '8589a8c9-77d1-4d72-b437-31f26ac07d82', '37ab0d60-b8fb-4f1b-b50e-e931e1db6dae', '02db64f7-d157-4fb6-823f-2576c371c4db', '212aee0c-2ba2-4ce7-a377-f8519e1cc851']",]

There are 31,188 houses within 1600m of any hospital in Exeter. First 5 osid: ['ed594210-3aa5-4f04-a1c2-479c54fcd967', '8589a8c9-77d1-4d72-b437-31f26ac07d82', '37ab0d60-b8fb-4f1b-b50e-e931e1db6dae', '02db64f7-d157-4fb6-823f-2576c371c4db', '212aee0c-2ba2-4ce7-a377-f8519e1cc851'].

Artifact name: houses_within_1600m_of_hospitals_map

Artifact description: A folium map showing all residential houses within 1600m of any hospital in Exeter, with the Exeter area boundary and hospital buffer for context.

Artifact data: houses_within_1600m_of_hospitals.html

host_agent

Here are the results for your query:

- There are 31,188 houses in Exeter that are within a 20-minute walk (1600m buffer) of a hospital.
- The search used the official Exeter area boundary, filtered for residential houses, and identified all hospitals in the area.
- The final map shows all these houses, the Exeter boundary, and the hospital buffer zones for context.

If you would like to view or download the map, let me know! I can also provide the list of house IDs or further details if needed.